

ASSIGNMENT NO.1

Algorithm:

1. Start.
2. Take the given integer array.
3. Initialize max with the smallest possible integer value (Integer.MIN_VALUE).
4. Initialize min with the largest possible integer value (Integer.MAX_VALUE).
5. Traverse each element of the array from the first element to the last.
6. For each element:
7. If the element is greater than max, update max.
8. If the element is smaller than min, update min.
9. After traversing the entire array, max contains the maximum element and min contains the minimum element.
10. Print the maximum and minimum values.
11. Stop.

Pseudocode:

BEGIN

Declare array arr

max $\leftarrow$ smallest possible integer

min $\leftarrow$ largest possible integer

FOR each element in arr DO

IF element > max THEN

max $\leftarrow$ element

END IF

IF element < min THEN

min $\leftarrow$ element

END IF

END FOR

PRINT "Maximum = ", max

PRINT "Minimum = ", min

END

Question 1 – Find Maximum and Minimum

Code:

```
public class AssignmentDSA1 {  
    public static void main(String[] args) {  
        int[] arr = {15,8,23,4,19,7};  
        int n = arr.length;  
  
        int Max = Integer.MIN_VALUE;  
        int Min = Integer.MAX_VALUE;  
  
        for(int i =0; i<n; i++){  
            if(arr[i] > Max) Max = arr[i];  
            if(arr[i] < Min) Min = arr[i];  
        }  
        System.out.println("Maximum = " + Max);  
        System.out.println("Minimum = " + Min);  
    }  
}
```

Time Complexity: $O(n)$

Algorithm:

1. Start.
2. Take the integer array.
3. Initialize Largest and SecondLargest to the smallest possible integer value.
4. Traverse the array from the first element to the last.
5. For every element:
6. If the element is greater than Largest:
7. Move the current Largest to SecondLargest.
8. Make the current element the new Largest.
9. Otherwise, if the element is greater than SecondLargest and different from Largest, update SecondLargest.
10. After traversing the complete array, SecondLargest contains the second-largest distinct element.
11. If there are fewer than two distinct elements, report that a second-largest element does not exist.
12. Stop.

Pseudocode:

```
BEGIN
Declare array arr
Largest  $\leftarrow -\infty$ 
SecondLargest  $\leftarrow -\infty$ 
FOR i  $\leftarrow$  0 TO n - 1
    IF arr[i] > Largest THEN
        SecondLargest  $\leftarrow$  Largest
        Largest  $\leftarrow$  arr[i]

    ELSE IF arr[i] > SecondLargest AND arr[i]  $\neq$  Largest THEN
        SecondLargest  $\leftarrow$  arr[i]
    END IF
END FOR
IF SecondLargest =  $-\infty$  THEN
    PRINT "Second largest element does not exist"
ELSE
    PRINT "Second Largest = ", SecondLargest
END IF
END
```

Question 2 – Find the Second Largest Element**Code:**

```
public class Code {
    public static void main(String[] args) {
        int[] arr = {12,5,8,20,15,20,7};
        int n = arr.length;
        int Largest = Integer.MIN_VALUE;
        int SecondLargest = Integer.MIN_VALUE;
        for (int i = 0; i < n; i++) {
            if(arr[i] > Largest){
                SecondLargest = Largest;
                Largest = arr[i];
            } else if(arr[i] > SecondLargest && arr[i] != Largest){
                SecondLargest = arr[i];
            }
        }
        System.out.println("Second Largest = "+ SecondLargest);
    }
}
```

Time Complexity: $O(n)$

Pseudocode:

```
BEGIN
  position  $\leftarrow$  0
  FOR i  $\leftarrow$  0 TO n - 1
    IF arr[i]  $\neq$  0 THEN
      arr[position]  $\leftarrow$  arr[i]
      position  $\leftarrow$  position + 1
    END IF
  END FOR
  WHILE position < n
    arr[position]  $\leftarrow$  0
    position  $\leftarrow$  position + 1
  END WHILE
  PRINT arr
END
```

Algorithm:

1. Start
2. Take the given array.
3. Initialize position = 0.
4. Traverse the array from left to right.
5. If the current element is non-zero:
6. Place it at arr[position].
7. Increment position.
8. After all non-zero elements are placed, fill the remaining positions with 0.
9. Print the array.
10. Stop.

Question 3 – Move All Zeros to the End

Code:

```
import java.util.Arrays;

public class Code {
    public static void main(String[] args) {
        int[] arr = {0, 5, 0, 3, 8, 0, 2};
        int n = arr.length;
        int position = 0;

        for (int i = 0; i < n; i++) {
            if(arr[i] != 0){
                arr[position] = arr[i];
                position++;
            }
        }
        while (position < arr.length) {
            arr[position] = 0;
            position++;
        }
        System.out.println(Arrays.toString(arr));
    }
}
```

Time Complexity: $O(n)$

Algorithm:

Step 1: Start

Step 2: Create an empty ArrayList<Integer> named queue.

Step 3: Display the menu:

1. Add Student
2. Submit Assignment
3. Search Student
4. Display Queue
5. Count Students
6. Exit

Step 4: Read the user's choice.

Step 5: If choice is 1:

- a. Read Student ID.
- b. Add the Student ID at the end of the queue.
- c. Display "Student added successfully."

Step 6: If choice is 2:

- a. Check whether the queue is empty.
- b. If the queue is empty, display "Queue is empty."
- c. Otherwise, remove the first student from the queue.
- d. Display the removed Student ID.

Step 7: If choice is 3:

- a. Read the Student ID to search.
- b. Check whether the Student ID exists in the queue.
- c. If found, display "Student is waiting."
- d. Otherwise, display "Student is not waiting."

Step 8: If choice is 4:

Display all students currently present in the queue.

Step 9: If choice is 5:

Find the size of the queue and display the total number of students currently waiting.

Step 10: If choice is 6:

Display "Program ended."

Step 11: If the choice is invalid:

Display "Invalid choice."

Step 12: Repeat Steps 3 to 11 until the user chooses 6.

Step 13: Stop.

Pseudocode:

```
BEGIN
  CREATE queue as ArrayList of Integer
  REPEAT
    DISPLAY:
      1. Add Student
      2. Submit Assignment
      3. Search Student
      4. Display Queue
      5. Count Students
      6. Exit
    READ choice
    IF choice = 1 THEN
      READ studentID
      ADD studentID to end of queue
      DISPLAY "Student added"
    ELSE IF choice = 2 THEN
      IF queue is empty THEN
        DISPLAY "Queue is empty"
      ELSE
        studentID = REMOVE first element from queue
        DISPLAY studentID + " submitted assignment"
      END IF
    ELSE IF choice = 3 THEN
      READ studentID
      IF studentID exists in queue THEN
        DISPLAY "Student is waiting"
      ELSE
        DISPLAY "Student is not waiting"
      END IF
    ELSE IF choice = 4 THEN
      DISPLAY queue
    ELSE IF choice = 5 THEN
      DISPLAY size of queue
    ELSE IF choice = 6 THEN
      DISPLAY "Program ended"
    ELSE
      DISPLAY "Invalid choice"
    END IF
  UNTIL choice = 6
END
```

Student Queue Management

A C-DAC classroom has a queue of students waiting to submit their assignments. Each student is represented by their Student ID, for example:

[105, 112, 108, 101, 115]

The faculty member can process one student at a time from the front of the queue.

However, some students may leave the queue before their turn.

You are asked to design a simple program to manage this queue using an ArrayList.

Your program should provide the following operations:

1. Add a student to the end of the queue.
2. Remove the student from the front of the queue after their assignment is submitted.
3. Display the current queue.
4. Search whether a particular Student ID is currently waiting.
5. Display the total number of students currently waiting.

Example

Initial queue: [105, 112, 108, 101, 115]

Student 105 submits.

Queue becomes: [112, 108, 101, 115]

Search:

Enter Student ID: 101

Output: Student 101 is waiting.

Current number of students: 4

Code:

```
import java.util.ArrayList;
import java.util.Scanner;

public class Code {

    public static void main(String[] args) {

        ArrayList<Integer> queue = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        int choice;

        do {
            System.out.println("\n----- Student Queue Management -----");
            System.out.println("1. Add Student");
            System.out.println("2. Submit Assignment");
            System.out.println("3. Search Student");
            System.out.println("4. Display Queue");
```

```

System.out.println("5. Count Students");
System.out.println("6. Exit");

System.out.print("Enter your choice: ");
choice = sc.nextInt();

switch (choice) {

    case 1:
        // Add student
        System.out.print("Enter Student ID: ");
        int studentId = sc.nextInt();

        queue.add(studentId);

        System.out.println(
            "Student " + studentId + " added to the queue."
        );
        break;

    case 2:
        // Remove student from front
        if (queue.isEmpty()) {

            System.out.println(
                "Queue is empty. No student to submit."
            );

        } else {

            int removedStudent = queue.remove(0);

            System.out.println(
                "Student " + removedStudent +
                " submitted the assignment."
            );
        }
        break;

    case 3:
        // Search student
        System.out.print("Enter Student ID to search: ");
        int searchId = sc.nextInt();

```

```

        if (queue.contains(searchId)) {

            System.out.println(
                "Student " + searchId + " is waiting."
            );

        } else {

            System.out.println(
                "Student " + searchId +
                " is not waiting."
            );
        }
        break;

    case 4:
        // Display queue
        System.out.println("Current Queue: " + queue);
        break;

    case 5:
        // Count students
        System.out.println(
            "Current number of students: " + queue.size()
        );
        break;

    case 6:
        // Exit
        System.out.println("Program ended.");
        break;

    default:
        System.out.println("Invalid choice.");
    }

} while (choice != 6);

sc.close();
}
}

```

Time Complexity: $O(n)$